Knowledge Distillation and Deployment of a Command-Parsing Model on a Local Cloud

Nirbhay, Roll No. 230103039 – https://github.com/nirbhay-iota/230103039

Summary and status

This project distils a command parser from a teacher (phi3.5 on Ollama, temperature 0, seed 42) into a 70M-parameter student (EleutherAI/pythia-70m) and serves it from a local k3d Kubernetes cluster with SeaweedFS object storage. The complete system is implemented in the repository (code, prompt, workers, training and evaluation scripts, service, Dockerfile, Kubernetes manifests, network files, CI workflow, 36 unit tests). **Only part of it was executed before the deadline.** Section 9 lists exactly what was and was not run. Measured results are given as measured; anything not run is stated as not run and is described only as future scope (Section 10), never as a result.

1. Architecture and decisions

A single-node k3d cluster (`cluster/k3d.yaml`) has three namespaces. **data**: SeaweedFS with buckets `datasets` and `models`. **pipeline**: an Indexed Job with two parallel teacher-labelling workers. **app**: the inference Deployment, Service, Ingress and HPA. The teacher runs on the host, outside the cluster, because a 3.8B model is easier to give CPU and memory there; Pods reach it as `host.k3d.internal`. The student is trained outside the cluster (`make train`); its files go to the `models` bucket and a GitHub Release, and the service image contains no weights (it downloads the model at start-up).

Decisions. (1) S3 credentials are generated per cluster into Kubernetes Secrets, so nothing secret is in Git. (2) `make up` rebuilds the system from files in the repository plus the model Releases. (3) Rolling updates use `maxUnavailable: 0, maxSurge: 1`, so a Ready Pod is removed only after its replacement is Ready. (4) The service loads its model in a background thread so that `/healthz` answers (liveness) while `/readyz` returns 503 and the Pod receives no traffic.

2. Task 1: object storage and parallel labelling

Executed. SeaweedFS ran in namespace `data` and both buckets were created, with `commands_pool.jsonl` uploaded (`results/setup/bootstrap.log`, `results/task1/seaweedfs.txt`). Two workers ran at the same time in namespace `pipeline` with `SHARD_INDEX` 0 and 1 and the same recorded start time, 2026-11-08T21:17:32Z (`results/task1/pipeline_worker_indexes.txt`). The teacher prompt is `prompts/teacher_prompt.txt`.

Sharding. Each worker takes the commands whose 0-based line number `n` satisfies `n % SHARD_COUNT == SHARD_INDEX` and writes its own object `datasets/teacher/shard-<i>.jsonl`. This static partition makes the shards disjoint by construction and together exhaustive, so there are no gaps and no repeats without coordination between workers (`worker/verify_shards.py` checks it). Round-robin by line number, rather than contiguous blocks, spreads differences in command length evenly over the workers. A worker flushes its object every 25 records and skips finished ids on restart. Assumption: with one Ollama server on one laptop the two workers share the same CPU, so the speed-up is below 2x.

Measured, partial. The labelling run was stopped before it finished. The locally saved snapshot holds **250 of 3,000** replies (125 per shard); 2,750 ids are missing from it, so the Task 1 requirement that every id appears exactly once is **not met** by the saved data. Cleaning the saved replies with `worker/clean.py` rules gives the counts below (`results/partial/summary.json`); the identity $250 = 126 + 0 + 1 + 11 + 0 + 112$ holds.

Input records	(a) not valid JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	Kept
250	126	0	1	11	0	112

Of the 250 raw replies, 124 (49.6%) parse as JSON and 112 (44.8%) survive all rules. Many rejected replies append explanatory text after the JSON; they were kept unedited as required. The SHA-256 of the partial cleaned file is `74c6e325f01a379e3ad33b206549f94c12e72166819dc7b0255711c34b7aef0f` (`results/partial/train.sha256`); it is **not** the final `train.sha256`, because the full dataset does not exist.

3. Task 2: training the student

Implemented, not executed. `training/train.py` fine-tunes Pythia-70M on the first 1,500 cleaned records as `Command: <text>\nJSON: <teacher reply><eos>` with the loss on the reply only. `training/config.yaml` records seed 42, learning rate 3e-4, batch size 16, maximum length 128 and 5 epochs for v1, and seed 43 with 6 epochs for v2. `make train`, `make train SUBSET=50` and `make train RUN=v2` read only this file; `training/package.py` builds a deterministic archive and its SHA-256 for the Release asset and `models/<run>/model.sha256`; `training/evaluate.py` and `training/eval_teacher.py` produce raw outputs for `score.py`. **No model was trained, no loss file, Release or validation score exists, so no accuracy figure is reported for the teacher, the untrained Pythia-70M or the student.**

Sequence-level vs logit-based distillation. In **sequence-level** distillation (Kim and Rush, 2016) the teacher decodes an output for every input and the student is trained with ordinary cross-entropy on that decoded text as if it were a label; the teacher's text is all the student sees. In **logit-based** distillation the student is trained to match the teacher's full probability distribution over the vocabulary at every position, typically by minimising a KL divergence between the two softmax outputs, so it also learns which alternatives the teacher considered likely. It is not possible here because (i) the teacher (Phi-3.5) and the student (Pythia, GPT-NeoX tokenizer) have different vocabularies, so their per-token distributions cannot be compared position by position, and (ii) Ollama's generation API returns text, not the full vocabulary distribution at each step, so the teacher's logits are not available to store. One limitation of this approach: the student can only copy what the teacher wrote, including its mistakes and its departures from the dataset's labelling conventions, and it sees a single greedy sample per command rather than the teacher's uncertainty, so the teacher's quality is a practical ceiling. The measured 49.6% JSON-parse rate of the saved teacher replies (Section 2) shows this matters: after cleaning only 44.8% of replies remain usable as training targets.

4. Task 3: the inference service

Implemented. `service/app.py` (FastAPI, port 8000) provides `POST /parse` (headers `X-Model-Version`, `X-Served-By`), `GET /healthz` and `GET /readyz` (503 until the model is loaded). Output is validated against `schema.json`; anything invalid becomes HTTP 500 `{"error": "invalid_model_output"}`, and `FAULT_INJECT=1` replaces the output with `not json` to demonstrate this. Configuration comes from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT` and `MAX_NEW_TOKENS`; credentials only from a Secret. Unit tests cover validation, the download filter and the API behaviour with a stub model.

Measured. The image build **did not complete**: the last attempt failed while downloading a dependency from `download.pytorch.org` with `Network is unreachable` (`results/setup/app_build_failed.log`). Consequently there is no image size, no 50-request memory test and no measured peak memory.

5. Task 4: Kubernetes deployment, scaling and reliability

Implemented, not exercised. `k8s/app/` holds the Deployment (2 replicas, requests 500m / 512Mi, limits 1 CPU / 1 GiB, liveness `/healthz`, readiness `/readyz`, zero-unavailable rolling updates), Service, Traefik Ingress for `/parse` and an HPA (1 to 2 replicas, 70% CPU). Scripts in `scripts/exp_*.sh` run the HPA, rolling update with rollback, Pod recovery, cold-start and load-balancing experiments. Because the image was never built, **none of these experiments was run**; there are no failed-request percentages, cold-start times or per-replica counts.

Limitation of the local cluster. The cluster is one node on one laptop. Deleting Pods would show that Kubernetes re-creates them, but it does not demonstrate tolerance to the loss of a machine, network partitions or zone failures; if the laptop or Docker stops, everything stops. Autoscaling from 1 to 2 replicas also competes with the load generator for the same CPU cores, so scaling behaviour on this setup is only indicative.

6. Task 5: subnetting and NACL

Allocation of `10.20.0.0/16` from the largest need to the smallest (working in `network/subnets.md`):

Subnet	Network	Prefix	Usable range	Broadcast	Usable hosts
app	10.20.0.0	/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public	10.20.4.0	/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	10.20.5.0	/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	10.20.5.128	/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

`network/nacl.json` (public subnet). Inbound: 100 deny all from 198.51.100.0/24; 110 allow tcp 80 and 120 allow tcp 443 from anywhere; 130 allow tcp 22 from 10.20.5.128/27; 140 allow tcp 1024-65535 from 10.20.0.0/22 (replies from the app subnet). Outbound: 100 allow tcp 8000 to 10.20.0.0/22; 110 allow tcp 1024-65535 to anywhere (replies to clients). **Measured:** `nacl_sim.py --check` gives 6 of 6 correct (100%) on the sample packets (`results/nacl_check.txt`); this does not establish the score on the hidden packets.

A NACL is stateless: it keeps no record of connections, so a reply is just another packet and needs its own rule. That is why replies to clients (outbound 110) and replies from the app subnet (inbound 140) are allowed explicitly on the ephemeral ports. Rules are checked from the lowest number up and the first match wins, so the deny for 198.51.100.0/24 must have a lower number (100) than the allows for ports 80 and 443 (110, 120); otherwise a packet from that range to port 443 would match the allow first and the block would never apply.

7. Task 6: reproducibility, testing and CI/CD

`make up` builds and deploys from a fresh clone and `make down` deletes the cluster; the README lists the commands. The workflow `.github/workflows/ci.yml` has the stages lint, unit tests, model evaluation from the Release asset, and the quality gate `score.py --min-validity 95`, plus a separate gitleaks job. *Measured locally* (`results/setup/`): 36 tests passed in 1.48 s; `ruff check` passed; gitleaks found no leaks in 13 commits (about 364,783 bytes). **No GitHub Actions run is recorded**, so there is no link to a passing run or to a run failing at the quality gate; the gate itself is untested end to end because no Release model exists.

8. Task 7: performance, cost and review

No load test was run, so p50, p95, throughput, error rate, per-Pod CPU and memory, and cold-start time are **not measured**.

Cost method (hypothetical rate card, `data/rate_card.json`; assumption: billing by limits for every hour a replica runs). CPU per replica-hour = 1 x 3.50 = 3.50 INR; memory = 1 x 0.45 = 0.45 INR; per replica 3.95 INR; two replicas 7.90 INR/hour. For a measured throughput of `r` requests/second, requests per hour = 3600 r and cost per 1,000 requests = 7.90 x 1000 / (3600 r) = 2.1944 / r INR. No `r` was measured, so no numeric cost per 1,000 requests is given.

SLO. Objective: 99% of requests succeed (HTTP 200 within 30 s); window: one load-test run of `N` requests. Error budget = 0.01 N requests; budget used = 100 F / (0.01 N) % for `F` failures. `N` and `F` were not measured.

Review (evidence is a measured value where one exists; otherwise the row says so):

Component	Evidence	Limitation found	Improvement proposed
Data and storage	250 of 3,000 replies saved; 112 kept (44.8%)	labelling is slow on CPU and was not finished	checkpointed workers already resume; run labelling longer or on a faster teacher host
Model quality	teacher replies parse as JSON in 124 of 250 (49.6%); no student score	teacher appends prose after JSON, so half the replies are unusable	tighten the prompt and stop sequences; accept JSON followed by text only if parsed before the prose
Latency and throughput	not measured (image never built)	no data	run `make loadtest` once the image builds
Resource utilisation	limits configured at 1 vCPU / 1 GiB per replica; 0 measurements	no peak-memory figure	run `scripts/memtest.sh`
Reliability and recovery	0 of 5 reliability experiments run	single node cannot show machine-level fault tolerance	run the experiment scripts; add a second node for real tolerance
Security and cost	gitleaks: 0 leaks in 13 commits; 7.90 INR/hour for two replicas	cost per 1,000 requests needs a throughput measurement	measure `r`, then use `scripts/cost_slo.py`

9. What was and was not executed

Executed and recorded	Not executed
k3d cluster, SeaweedFS, buckets, command-pool upload	completion of the 3,000-command labelling and the final `train.jsonl`
two parallel workers with shard indexes 0 and 1 (partial output, 250 replies)	training of v1 and v2, loss file, model Releases
cleaning rules applied to the partial output	validation scores of teacher, untrained model and student
NACL simulator, 6 of 6 sample packets	service image build, memory test, all Task 4 experiments
36 unit tests, ruff, gitleaks (local)	GitHub Actions runs, load test, latency and cost figures

10. Future scope: expected outputs once the remaining steps are run

These are acceptance criteria and expectations, not results.

- **Task 1:** all 3,000 ids appear once across the two shards (`make verify-shards` prints OK); `make clean-data` writes `datasets/v1/train.jsonl` and a checksum that re-running the cleaner reproduces; the counts table then sums to 3,000.
- **Task 2:** `make train SUBSET=50` runs to the end; `training/loss.csv` has one row per epoch with non-increasing loss; the archive hash equals `models/v1/model.sha256`; the untrained Pythia-70M is expected to score near 0% JSON validity and the trained student far higher, with the target on the hidden set of at least 95% validity.
- **Task 3:** an image of at most 2 GB; 50 sequential requests at 1 CPU / 1 GiB complete with no OOM kill and a reported peak memory; `FAULT_INJECT=1` returns HTTP 500 with the exact error body.
- **Task 4:** the Service has no ready endpoints while the model loads; the HPA goes from 1 to 2 replicas under load and back; a rolling update to v2 fails at most 0.5% of requests and `rollout undo` returns `X-Model-Version: v1`; a deleted Pod is replaced; three cold starts are reported with their median; each of two replicas serves at least 30% of requests.
- **Task 6:** one CI run on the Release model passes and one run on the untrained model fails at the quality gate.
- **Task 7:** error rate at most 1% and p95 at most 5,000 ms; the cost per 1,000 requests then follows from the formula in Section 8.

References

Kim and Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016. Bastianelli et al., *SLURP*, EMNLP 2020 (CC BY 4.0). EleutherAI Pythia-70M model card (Apache 2.0). Phi-3.5-mini-instruct on Ollama. k3d, SeaweedFS, Kubernetes, Locust and gitleaks documentation.